It doesn't work because when the function is invoked indirectly, the @ reference is lost. Normally in CoffeeScript you use the fat arrow `=>` to lexically bind the @ reference for a function. In this case, though, you don't want to lexically bind the @, but dynamically bind it.

call and apply

Luckily, the `Function` prototype has two methods, `call` and `apply`, that can help you in the rare instances where neither the arrow (`->`) nor fat arrow (`=>`) provide what you need. Both `call` and `apply` allow you to invoke any function with a specific `this` bound.

With `call` you can invoke a function with a specific `this` and a set number of arguments. Here's a modified example:

```
airplane =
  startEngine: -> 'Engine started!'

withRunningEngine = (first, second) ->
  @startEngine()
  "#{first} then #{second}"

withRunningEngine 'Take-off', 'Fly'
# Object #<Object> has no method 'startEngine'

withRunningEngine.call airplane, 'Take-off', 'Fly'
'Take-off then Fly'
```

With `apply` you can invoke a function with a specific `this` and an array of arguments:

```
withRunningEngine.apply airplane, ['Take-off', 'Fly']
'Take-off then Fly'
```

Although the addition of the fat arrow to CoffeeScript removes many of the common uses of `call` and `apply` found in JavaScript, there are still occasions, such as now, when you'll find them useful. Using `apply` you can create new versions of `before`, `after`, and `around` that can play nicely with methods. These new versions are shown in the next listing.

Listing 6.6. `Before`, `after`, and `around` with function binding